

DE Portfolio Notes (SQL + Python ETL)

Keyword: DE-PORTFOLIO-SAI

DE-PORTFOLIO-SAI

SQL Portfolio Structure + Python ETL Sample (CSV → MySQL) + Advanced Staging→Final Incremental Load

```
=====
1) SQL PORTFOLIO (GitHub) STRUCTURE
=====
```

Create one main repo:

```
sql-portfolio/
```

Recommended structure:

```
sql-portfolio/
├── README.md
├── 01-sales-report/
│   ├── README.md
│   ├── schema.sql
│   ├── insert_data.sql
│   ├── queries.sql
│   ├── outputs/
│   │   ├── q1_total_sales.png
│   │   ├── q2_category_sales.png
│   │   └── q3_city_revenue.png
│   └── dataset/
│       └── sales_data.csv
├── 02-employee-analytics/
│   ├── README.md
│   ├── schema.sql
│   ├── insert_data.sql
│   ├── queries.sql
│   ├── outputs/
│   └── dataset/
└── 03-banking-transactions-report/
    ├── README.md
    ├── schema.sql
    ├── insert_data.sql
    ├── queries.sql
    ├── outputs/
    └── dataset/
```

Architecture (for each project)

DATA → DATABASE → QUERY → RESULT → INSIGHTS

- 1) Dataset: CSV/Excel
- 2) Schema: tables + datatypes
- 3) Load: import or insert scripts
- 4) Queries: 10-20 queries
- 5) Output Proof: screenshots of results
- 6) Insights: 3-5 key findings

```
=====
2) PROJECT 1: SALES REPORT (SQL)
=====
```

```
schema.sql
```

```
-----
CREATE DATABASE IF NOT EXISTS portfolio_db;
USE portfolio_db;
```

```
DROP TABLE IF EXISTS sales;
```

```
CREATE TABLE sales (  
    order_id INT,  
    order_date DATE,  
    customer_name VARCHAR(50),  
    city VARCHAR(30),  
    product VARCHAR(30),  
    category VARCHAR(20),  
    quantity INT,  
    unit_price DECIMAL(10,2),  
    payment_mode VARCHAR(20)  
);
```

```
insert_data.sql
```

```
-----
```

```
USE portfolio_db;
```

```
INSERT INTO sales VALUES
```

```
(101, '2025-01-05', 'Ravi', 'Hyderabad', 'Headphones', 'Electronics', 2, 1200, 'UPI'),  
(102, '2025-01-06', 'Sita', 'Vijayawada', 'Shoes', 'Fashion', 1, 1800, 'Card'),  
(103, '2025-01-06', 'Arjun', 'Hyderabad', 'T-Shirt', 'Fashion', 3, 600, 'Cash'),  
(104, '2025-01-07', 'Kiran', 'Kavali', 'Power Bank', 'Electronics', 1, 1500, 'UPI'),  
(105, '2025-01-07', 'Meena', 'Nellore', 'Mixer', 'Home', 1, 3200, 'Card'),  
(106, '2025-01-08', 'Rahul', 'Kavali', 'Headphones', 'Electronics', 1, 1200, 'UPI'),  
(107, '2025-01-09', 'Divya', 'Hyderabad', 'Shoes', 'Fashion', 2, 1800, 'UPI'),  
(108, '2025-01-10', 'Naveen', 'Nellore', 'Bottle', 'Home', 4, 250, 'Cash');
```

```
queries.sql
```

```
-----
```

```
USE portfolio_db;
```

```
-- 1) Total sales amount
```

```
SELECT SUM(quantity * unit_price) AS total_sales  
FROM sales;
```

```
-- 2) Category-wise revenue
```

```
SELECT category, SUM(quantity * unit_price) AS revenue  
FROM sales  
GROUP BY category  
ORDER BY revenue DESC;
```

```
-- 3) City-wise revenue
```

```
SELECT city, SUM(quantity * unit_price) AS revenue  
FROM sales  
GROUP BY city  
ORDER BY revenue DESC;
```

```
-- 4) Top selling product by quantity
```

```
SELECT product, SUM(quantity) AS total_qty  
FROM sales  
GROUP BY product  
ORDER BY total_qty DESC  
LIMIT 1;
```

```
-- 5) Payment mode usage
```

```
SELECT payment_mode, COUNT(*) AS orders, SUM(quantity * unit_price) AS amount  
FROM sales  
GROUP BY payment_mode  
ORDER BY amount DESC;
```

```
=====
```

```
3) PYTHON ETL SAMPLE (CSV → CLEAN → LOAD INTO MYSQL)
```

```
=====
```

```
Install:
```

```
pip install pandas mysql-connector-python
```

Sample CSV file: sales_raw.csv

```
-----  
order_id,order_date,customer_name,city,product,quantity,unit_price,payment_mode  
101,2025-01-05,Ravi,Hyderabad,Headphones,2,1200,UPI  
102,2025-01-06,Sita,Vijayawada,Shoes,1,1800,Card  
103,2025-01-06,Arjun,Hyderabad,T-Shirt,3,600,Cash  
103,2025-01-06,Arjun,Hyderabad,T-Shirt,3,600,Cash  
104,2025-01-07,Kiran,Kavali,Power Bank,1,1500,UPI  
105,2025-01-07,,Nellore,Mixer,1,3200,Card  
106,2025-01-08,Rahul,Kavali,Headphones,one,1200,UPI  
107,2025-01-09,Divya,Hyderabad,Shoes,2,1800,UPI  
108,2025-01-10,Naveen,Nellore,Bottle,4,,Cash
```

Python script: etl_load_sales.py

```
-----  
import pandas as pd  
import mysql.connector  
from datetime import datetime  
  
CSV_FILE = "sales_raw.csv"  
  
DB_CONFIG = {  
    "host": "localhost",  
    "user": "root",  
    "password": "YOUR_PASSWORD",  
    "database": "portfolio_db"  
}  
  
TABLE_NAME = "sales_clean"  
  
def clean_sales_data(df: pd.DataFrame) -> pd.DataFrame:  
    df = df.drop_duplicates()  
    df["customer_name"] = df["customer_name"].fillna("Unknown")  
    df["order_date"] = pd.to_datetime(df["order_date"], errors="coerce").dt.date  
    df["quantity"] = pd.to_numeric(df["quantity"], errors="coerce").fillna(1).astype(int)  
    df["unit_price"] = pd.to_numeric(df["unit_price"], errors="coerce").fillna(0).astype(float)  
  
    for col in ["customer_name", "city", "product", "payment_mode"]:  
        df[col] = df[col].astype(str).str.strip()  
  
    return df  
  
def create_table(cursor):  
    cursor.execute(f"""  
    CREATE TABLE IF NOT EXISTS {TABLE_NAME} (  
        order_id INT,  
        order_date DATE,  
        customer_name VARCHAR(50),  
        city VARCHAR(30),  
        product VARCHAR(30),  
        quantity INT,  
        unit_price DECIMAL(10,2),  
        payment_mode VARCHAR(20)  
    )  
    """)  
  
def load_data_to_mysql(df: pd.DataFrame, cursor):  
    insert_query = f"""  
    INSERT INTO {TABLE_NAME}  
    (order_id, order_date, customer_name, city, product, quantity, unit_price, payment_mode)  
    VALUES (%s,%s,%s,%s,%s,%s,%s,%s)  
    """  
    cursor.executemany(insert_query, list(df.itertuples(index=False, name=None)))  
  
def main():  
    df_raw = pd.read_csv(CSV_FILE)  
    df_clean = clean_sales_data(df_raw)  
    conn = mysql.connector.connect(**DB_CONFIG)
```

```

        cursor = conn.cursor()
        create_table(cursor)
        load_data_to_mysql(df_clean, cursor)
        conn.commit()
        cursor.close()
        conn.close()

    print("Data loaded successfully into:", TABLE_NAME)

if __name__ == "__main__":
    main()

```

```

=====
4) ADVANCED DATA ENGINEERING ETL
CSV → STAGING → FINAL (INCREMENTAL LOAD + LOGS)
=====

```

Python script: etl_staging_to_final.py (key components)

-
- sales_staging: temporary clean load
 - sales_final: final table with PRIMARY KEY (order_id)
 - etl_run_log: tracks each run status and counts
 - Uses MySQL UPSERT with ON DUPLICATE KEY UPDATE

Core idea:

- 1) Read CSV
- 2) Clean
- 3) TRUNCATE staging
- 4) Load staging
- 5) UPSERT staging -> final
- 6) Log SUCCESS/FAILED with row counts

Portfolio proof:

- terminal success output screenshot
- final table screenshot
- log table screenshot

Keyword to continue later: DE-PORTFOLIO-SAI